TECHNICAL NOTE · MEDICAL INFORMATION RETRIEVAL · RE-RANKING

When the Re-ranker Makes Things Worse

Fine-tuning a cross-encoder for medical evidence retrieval: an ablation showing that label quality, not model or learning rate, decides the outcome — and what it costs to serve.

Mustapha HAMDI · October 2026 · Code, configs and run files: *clinical-evidence-retrieval* (GitHub link added on publication)

TL;DR

- An off-the-shelf MS MARCO cross-encoder placed after a strong hybrid retriever **lowers** nDCG@10 on SciFact (−0.028, p = 0.055).
- A first fine-tuning on our own hard negatives never beats the base model. Changing the learning rate does not help either; switching to a listwise loss helps a little.
- The cause is the training labels. Trained on 608 clean in-domain claims only, with a listwise loss, the re-ranker fixes the regression: +0.027 nDCG@10 vs the off-the-shelf model on test (p = 0.002), in 3 minutes of GPU training.
- No re-ranker beats the hybrid first stage significantly on these benchmarks; on NFCorpus the first stage caps what any re-ranker can do (Recall@100 = 0.301).
- Serving: in fp16 on a T4 the model re-ranks 20 passages in 27 ms p95, inside a 100 ms budget; a laptop CPU needs ≈ 1.0 s even in int8. At GPU speed, tokenization becomes half of the cost.

1. Why re-ranking matters in medical retrieval

A clinical assistant that answers from guidelines and studies is only as good as the passages it retrieves: if the right recommendation is not in the context, no language model can recover it, and if a weak source is ranked first, it will be cited confidently. Production retrieval is therefore usually a **two-stage** pipeline (Figure 1). A fast first stage searches the whole collection and returns about a hundred candidates; a slower, more accurate model then *re-ranks* the best of them.

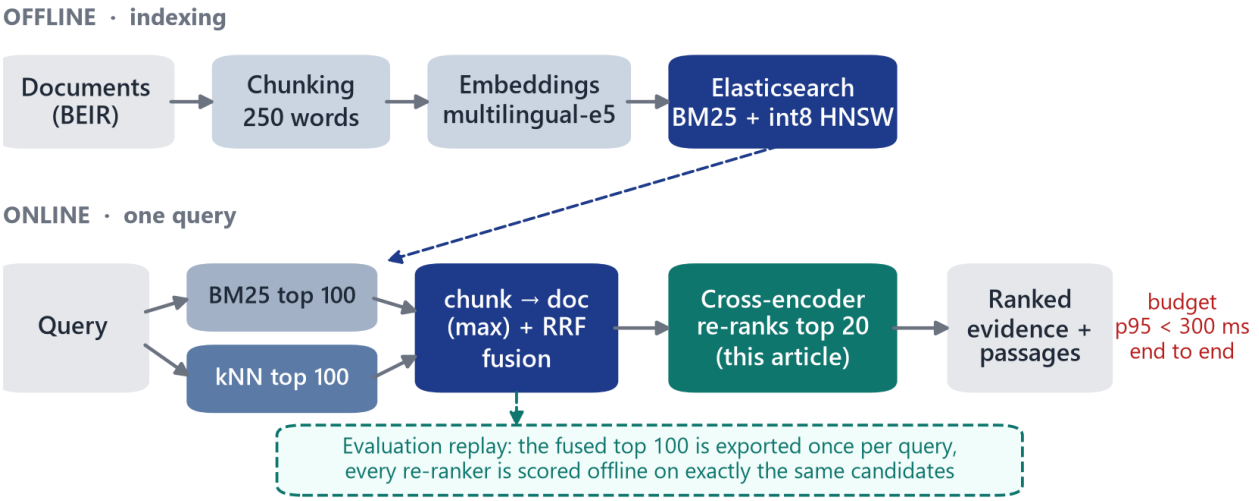


Figure 1. The pipeline studied here. Offline, documents are chunked, embedded and indexed once. Online, BM25 and vector search run in parallel, their lists are merged with Reciprocal Rank Fusion, and a cross-encoder re-orders the top 20. For evaluation, the fused top 100 is exported once and every re-ranker is replayed on exactly the same candidates.

Bi-encoder vs cross-encoder, in one paragraph

A **bi-encoder** (the embedding model of the first stage) turns the query and each passage into vectors *separately*; passages are encoded once at indexing time and search is a nearest-neighbour lookup, which scales to millions of documents. A **cross-encoder** reads the query and one passage *together*, so every word of the query can attend to every word of the passage. It is much more accurate at judging relevance, but it must run once per (query, passage) pair at query time: affordable for 20 candidates, not for a whole corpus. That is why it sits in second position.

Off-the-shelf cross-encoders are typically trained on MS MARCO, a large set of web search questions. The question this note answers is practical: *does such a model help on medical evidence, can we make it better with our own data, and what does it cost to run?*

2. Setup: data, first stage and an evaluation you can trust

Two public benchmarks

NFCorpus (Boteva et al., 2016): nutrition and medical questions written for the public, linked to PubMed articles; about 40 relevant documents per query, with graded relevance. **SciFact** (Wadden et al., 2020): scientific *claims* ("Vitamin D supplementation reduces fracture risk") paired with the abstracts that support or refute them; usually a single relevant abstract. Both come from the BEIR benchmark (Thakur et al., 2021) with official train/dev/test splits. Test sets have 323 and 300 queries; indexes hold 4,811 and 6,428 chunks.

A hybrid first stage

Documents are split into chunks of about 250 words and indexed in Elasticsearch twice over: as text for BM25 (keyword matching) and as vectors from *multilingual-e5-base* (semantic matching, Wang et al., 2024). For each query both lists are retrieved in one round trip, each chunk is mapped to its document, and the two rankings are merged with Reciprocal Rank Fusion (Cormack et al., 2009), which only uses ranks and therefore needs no score calibration:

$$\text{score}(d) = \sum_{s \in \{\text{BM25, kNN}\}} \frac{1}{k + \text{rank}_s(d)}, \quad k = 60$$

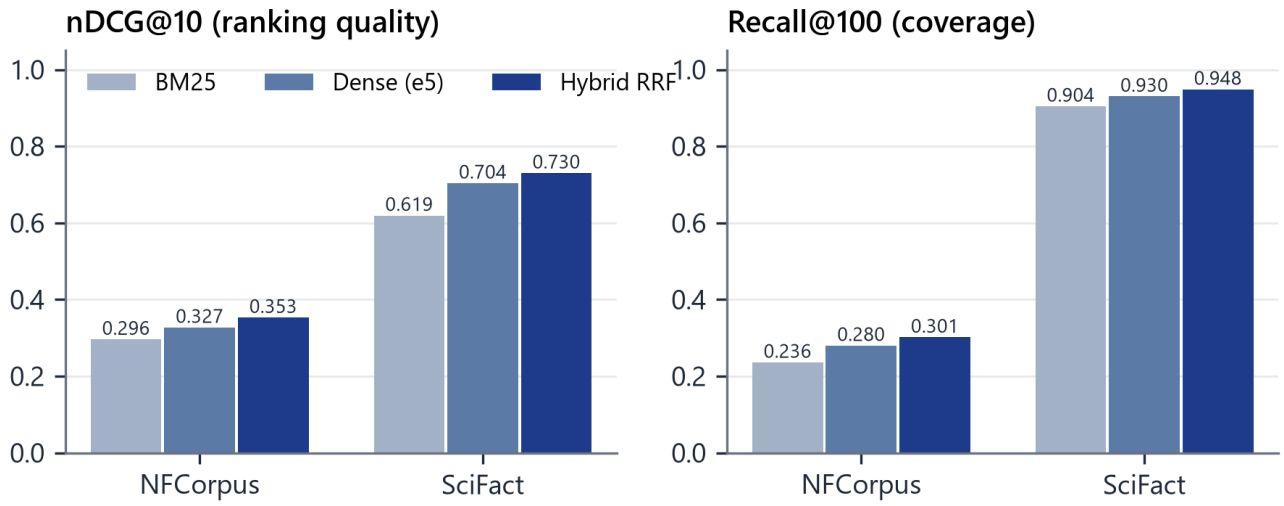


Figure 2. First-stage results on the test sets. Hybrid fusion beats both of its components on both datasets and both metrics; it is the baseline every re-ranker has to beat.

Reading the metrics

nDCG@10 scores the first ten results: a relevant document counts more the higher it is ranked (logarithmic discount), and the score is normalised by the best possible ranking, so 1.0 is perfect.

Recall@100 is the share of relevant documents present anywhere in the top 100: what the second stage could possibly promote. **MRR@10** looks only at the rank of the first relevant result.

$$\text{nDCG@10} = \frac{1}{\text{IDCG}} \sum_{i=1}^{10} \frac{\text{rel}_i}{\log_2(i+1)}$$

Protocol

- **Replay, not re-run.** The fused top 100 of every query is exported once; every re-ranker, depth and checkpoint is then scored on identical candidates, without Elasticsearch. Replaying the export without re-ranking reproduces the first-stage runs exactly, a built-in consistency check.
- **Validation for decisions, test once.** Depths, checkpoints and model choices are made on NFCorpus dev and on 200 held-out SciFact training claims; each test set is evaluated once per final system, with the decision rule written down before looking.
- **Paired significance.** Two systems are compared query by query with a paired permutation test (10,000 permutations; Smucker et al., 2007) and 95% bootstrap confidence intervals, not by comparing means.

> **Why a permutation test?**
> With 300 queries, a difference of 0.01 in mean nDCG@10 can easily be luck: a handful of queries flipping decides it. The permutation test asks how often a difference at least this large appears if we randomly swap the two systems' scores within each query. A small p-value means the observed gap is unlikely under "no difference". It is paired, so the large variance between easy and hard queries cancels out.

3. Surprise #1: the off-the-shelf re-ranker hurts

We first re-rank with *cross-encoder/ms-marco-MiniLM-L-6-v2*, a popular 22M-parameter model. The depth (how many fused candidates to re-score) is tuned on validation (Figure 3, right): re-ranking more never helps, and on SciFact every depth is *below* the first stage (0.730 at top 20 vs 0.750 without re-ranking). We keep top 20 and evaluate once on test (Table 1).

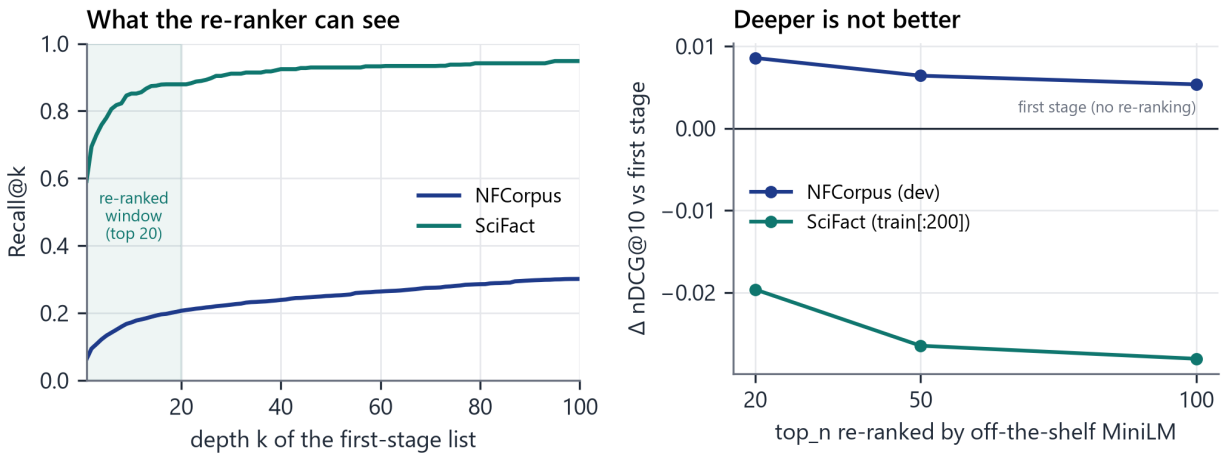


Figure 3. Left: share of relevant documents found within the first k candidates (test). On NFCorpus only 21% of relevant documents are in the re-ranked window and 30% in the top 100: a hard ceiling for any re-ranker. SciFact reaches 88% at 20. Right: effect of the re-ranking depth for the off-the-shelf model on the validation sets.

Table 1. Off-the-shelf re-ranking on test (top 20 re-ranked).

Test, nDCG@10	BM25	Dense	Hybrid RRF	+ MiniLM (MS MARCO)	Δ vs hybrid
NFCorpus	0.296	0.327	0.353	0.361	+0.009 (p = 0.156)
SciFact	0.619	0.704	0.730	0.701	−0.028 (p = 0.055)

Why would a strong re-ranker make things worse? Two reasons that compound. First, the baseline is already high: the BEIR authors report gains of about +0.02 on these two datasets for an MS MARCO cross-encoder re-ranking *BM25*, but our hybrid first stage is already +0.111 above BM25 on SciFact. Second, the task is different. MS MARCO teaches "does this passage answer the question?"; SciFact asks "is this the evidence for or against this claim?". Claims are declarative, often negated ("... does not cause ..."), and the relevant abstract may well *contradict* them, something a question-answering model is not trained to reward. Negated claims indeed lose the most in our per-query analysis.

4. Surprise #2: fine-tuning on our own data does not help either

The natural fix is to fine-tune the re-ranker on the training splits of the two datasets, with **hard negatives** mined by our own first stage: non-relevant documents that the retriever ranks high, precisely the mistakes the re-ranker must learn to correct (Figure 4). On NFCorpus, whose judgments are incomplete, the top three unjudged documents are not used as negatives, because they are often relevant but unlisted.

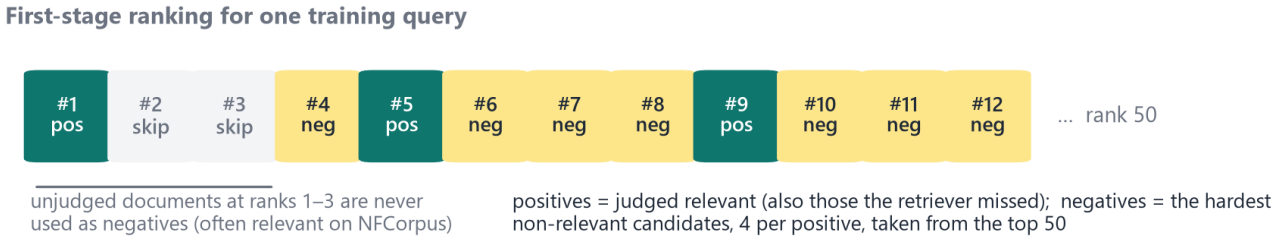


Figure 4. Building training pairs for one query from the first-stage ranking (illustrative).

The resulting set has 93,909 (query, passage, label) pairs, 20% positive. Training queries whose text is identical to a test or validation query were removed (a few exist across BEIR splits). We train with the standard pointwise binary cross-entropy:

$$\mathcal{L}_{\text{pointwise}} = -\,y \log \sigma(s_{q,d}) - (1-y)\log\left(1-\sigma(s_{q,d})\right)$$

where s is the model's score for the pair and y its 0/1 label. Figure 5 shows what happens on validation: the model first *forgets* (both datasets drop sharply), then NFCorpus slowly recovers to the base level while SciFact never does. No checkpoint beats the starting model, so the training script, which keeps the best validation checkpoint and starts from the base model, simply returns the base model.

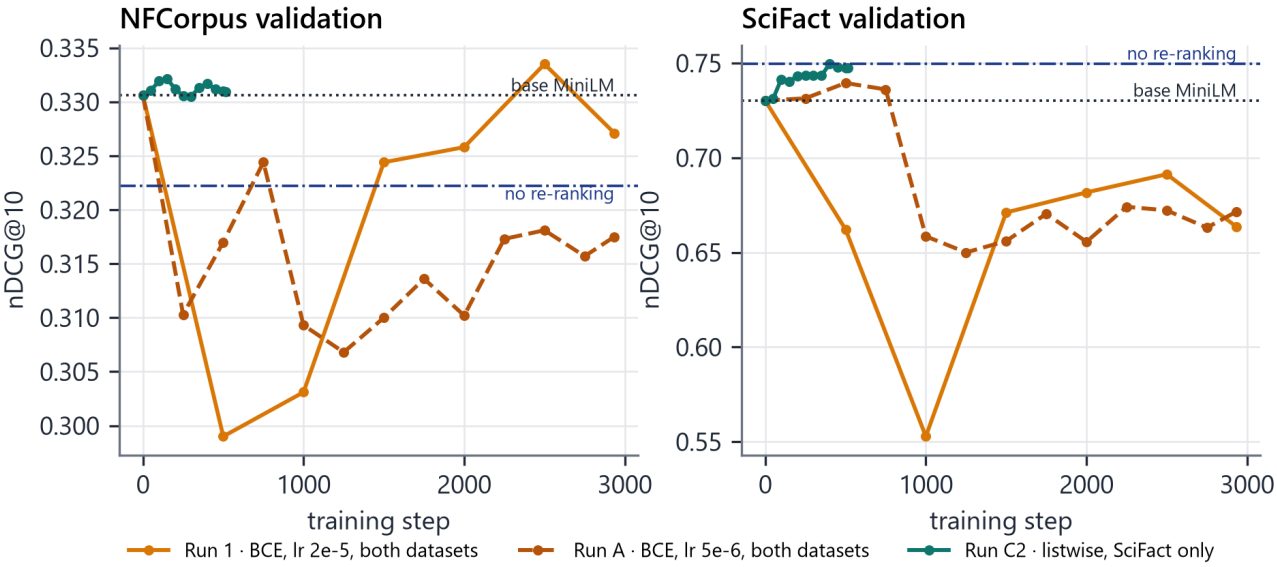


Figure 5. Validation nDCG@10 during fine-tuning (Kaggle T4). Dotted: the off-the-shelf model (step 0). Dash-dotted: the first stage without re-ranking. Pointwise runs (orange) degrade, with a high or a low learning rate; the listwise run on SciFact only (green) improves and stays there.

5. The ablation: what actually matters

A failed run has many possible causes. We changed one factor at a time, with the same data, validation sets and selection rule (Figure 6).

- **Learning rate (run A, 4× lower).** The collapse on SciFact is only delayed, from step 500 to about 750. Not the cause.

- **Loss (run B).** A re-ranker only needs to *order* the documents of one query, not to predict an absolute probability. The listwise ("localised contrastive") loss of Gao et al. (2021) compares one positive against seven hard negatives of the same query with a softmax:

$$\mathcal{L}_{\text{listwise}} = -\log \frac{\exp(s_{q,d^+})}{\exp(s_{q,d^+}) + \sum_{j=1}^{7} \exp(s_{q,d_j^-})}$$

- With the same data, run B is better than the pointwise runs (SciFact +0.013 over the base model) but still loses on NFCorpus (−0.006). Helps, but not enough.
- **Data (runs C1, C2).** Trained on NFCorpus alone, the listwise model never improves, not even on NFCorpus. Trained on SciFact alone (608 claims, 516 steps, 3 minutes), it improves SciFact by +0.019 and leaves NFCorpus untouched (+0.001).

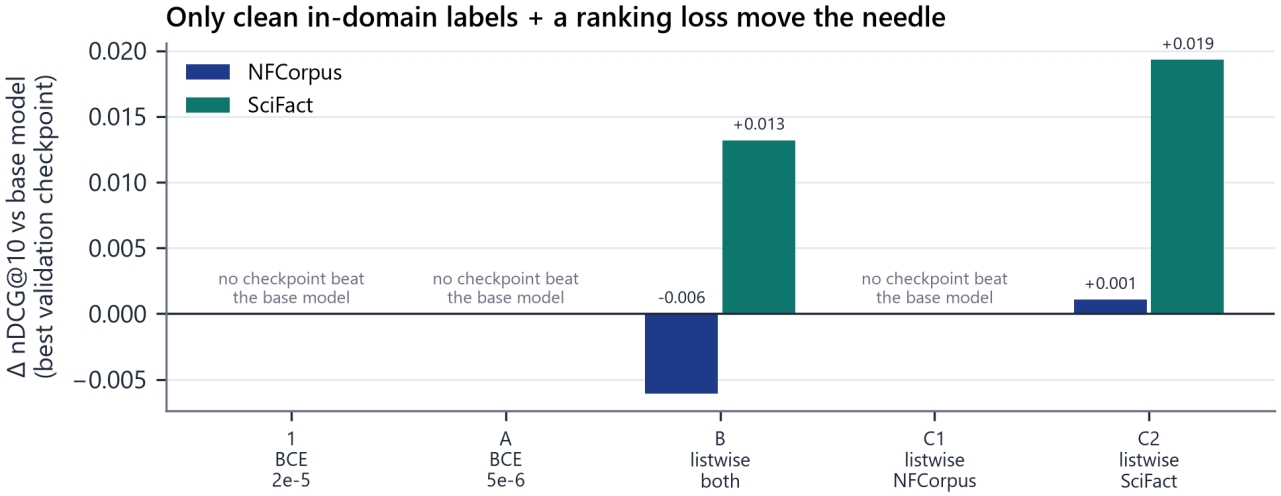


Figure 6. Best validation checkpoint of each fine-tuning run, relative to the off-the-shelf model. Runs 1, A and C1 never beat it. Only the listwise loss on clean in-domain labels (C2) improves without hurting the other dataset.

The diagnosis is the labels. NFCorpus judgments are derived from citation links between a health website and PubMed: about forty "relevant" documents per query, many only loosely on topic. For a model that already knows how to rank, these broad labels contain little new information, and they pull it away from the precise evidence matching SciFact needs. Mixing them in degrades SciFact; removing them is what works. More data was not better: 83,604 weak pairs did harm, 608 clean ones helped.

6. Test results, with uncertainty

The fine-tuned model, called C2, was then evaluated once on both test sets with the rule fixed beforehand: keep it if it is not significantly worse than the first stage on either dataset and it beats the off-the-shelf model on SciFact. Figure 7 shows each difference with its 95% confidence interval.

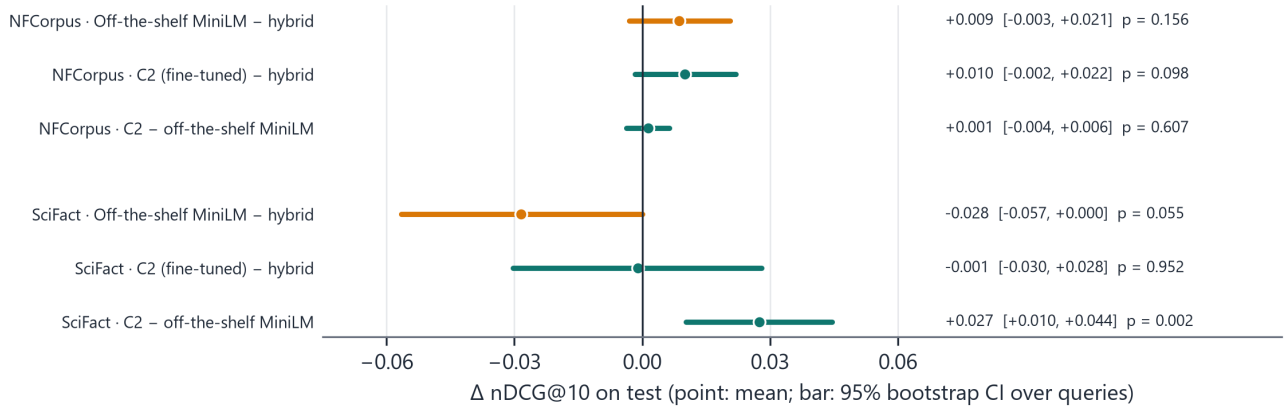


Figure 7. Paired differences in nDCG@10 on test. An interval that crosses zero means the data cannot rule out "no difference". p-values from paired permutation tests.

Table 2. Final comparison on test (top 20 re-ranked).

Test	Hybrid RRF	+ MiniLM	+ C2 (fine-tuned)	C2 vs MiniLM	C2 vs hybrid
NFCorpus nDCG@10	0.353	0.361	0.363	+0.001 (p = 0.607)	+0.010 (p = 0.098)
SciFact nDCG@10	0.730	0.701	0.729	**+0.027 (p = 0.002)**	−0.001 (p = 0.952)
NFCorpus MRR@10	0.560	0.588	0.586		
SciFact MRR@10	0.697	0.672	0.699		

Two conclusions, and it matters to state both. Fine-tuning on a small clean in-domain set **does fix** the domain mismatch of the off-the-shelf model, with an interval well clear of zero. But **no re-ranker beats the hybrid first stage significantly**: on NFCorpus the gain is about +0.01 with an interval touching zero, on SciFact C2 is level with the first stage. With 300 queries, effects of this size are below what the protocol can detect, and on NFCorpus the re-ranker can only reorder the 21% of relevant documents that the first stage put in its top 20.

7. What it costs to serve

A re-ranker earns its place only if it fits the latency budget: here p95 < 300 ms end to end, of which about 100 ms for re-ranking. We measured C2's architecture on a laptop CPU (exported to ONNX, fp32 and dynamically quantized int8) and on an NVIDIA T4 GPU (fp32 and fp16), with real passages, warm-up excluded (Figure 8).

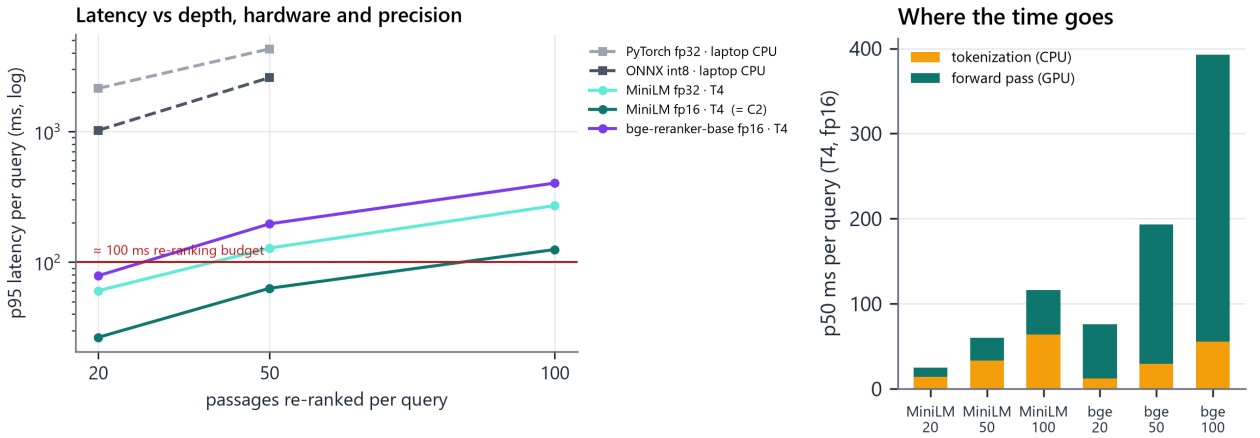


Figure 8. Left: p95 latency per query against the number of passages re-ranked (log scale). Right: on the GPU in fp16, tokenization on the CPU is as large as the model itself.

- **GPU fp16 fits with margin**: 27 ms p95 for 20 passages, 63 ms for 50; fp16 is 2–4× faster than fp32 on the T4 (60 ms at 20).
- **Int8 on CPU** keeps ranking quality on validation and cuts the model 4× in size, but a 15 W laptop still needs ≈ 1022 ms p95 for 20 passages (2140 ms in fp32): a fallback, not a serving target.

- **The larger bge-reranker-base** fits only at depth 20 in fp16 (79 ms) and costs about 3× MiniLM: it would need a clear quality gain to justify it.
- **Tokenization becomes the bottleneck**: 14 of 25 ms at depth 20. Passages can be tokenized once at indexing time and stored with the chunk, so only the query is tokenized online.

8. Lessons

- **Measure the re-ranker against the first stage, not against nothing.** A strong hybrid retriever leaves little room, and an off-the-shelf model can lose it.
- **Label quality beats label quantity.** 608 clean, task-matched examples helped; 83,604 broad ones hurt. In a clinical setting this argues for small clinician-validated sets over large weak ones.
- **Use a ranking objective.** A listwise loss over a query's own hard negatives fits what a re-ranker does.
- **Ablate one factor at a time and keep failed runs.** Without runs A, B and C1, the conclusion would have been "lower the learning rate", which is wrong.
- **Recall is the ceiling.** When the first stage misses 70% of the relevant documents, the next gains come from query processing and candidate generation, not from a better re-ranker.
- **Precision and tokenization are serving levers.** fp16 on GPU, int8 on CPU, and pre-tokenized passages.

Limitations

Two small English benchmarks; labels from benchmark judgments, not clinicians; one training seed per run; laptop-scale indexes (thousands of chunks), so first-stage latency here says nothing about 100M-document behaviour; GPU throughput is extrapolated, not load-tested. French queries and guideline documents are the next steps of the project.

Reproducibility

Every number and figure in this note is generated from run files by `docs/article/make_figures.py`. Commands: `cer candidates` (export first-stage candidates), `cer rerank` (replay a re-ranker), `scripts/train_reranker.py` (fine-tuning, listwise or pointwise), `cer export-onnx` and `scripts/bench_reranker*.py` (serving cost).

References

Boteva, V., Gholipour, D., Sokolov, A., Riezler, S. (2016). A Full-Text Learning to Rank Dataset for Medical Information Retrieval. *ECIR*.

Cormack, G. V., Clarke, C. L. A., Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*.

Gao, L., Dai, Z., Callan, J. (2021). Rethink Training of BERT Rerankers in Multi-Stage Retrieval Pipeline. *ECIR*.

Smucker, M. D., Allan, J., Carterette, B. (2007). A Comparison of Statistical Significance Tests for Information Retrieval Evaluation. *CIKM*.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., Gurevych, I. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *NeurIPS Datasets and Benchmarks*.

Wadden, D., Lin, S., Lo, K., Wang, L. L., van Zuylen, M., Cohan, A., Hajishirzi, H. (2020). Fact or Fiction: Verifying Scientific Claims. *EMNLP*.

Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., Wei, F. (2024). Multilingual E5 Text Embeddings: A Technical Report. *arXiv:2402.05672*.